

An Introductory Guide to Inform 7

Created by Jason Boyd, Ryerson University, 2021

Version 1 (March 4, 2021): Incorporating material from *Writing with Inform* (WWI) up to and including Chapter 17.

[Preface](#)

[1. The Inform IDE](#)

[2. Titling your work & division headings](#)

[3. Creating Rooms, Connections, and Things](#)

[3.1 Things in Parts \(Composite Things\)](#)

[4. Describing Rooms and Things: Authored and Generated Descriptions](#)

[4.1 Descriptions of Rooms](#)

[4.2 Descriptions of Things](#)

[4.3 Scenery, Backdrops, and Undescribed Things](#)

[4.4 Changing Descriptions](#)

[4.5 Sounds and Smells, Taste and Touch](#)

[5. Setting the States of Rooms and Things: Properties](#)

[5.1 Spaces](#)

[5.2 Things, Supporters, Containers](#)

[5.3 Vehicles](#)

[5.4 Doors, Locks and Keys](#)

[5.5 Devices](#)

[6. Living Creatures](#)

[6.1 Movement of non-player characters/creatures](#)

[7. Responding to Player Input](#)

[7.1 Basic Rules \(Instead, Before, After, If, Now\)](#)

[7.2 The Player Character \(Placing, Appearance and Inventory\)](#)

[7.3 Interacting with Non-Player Characters \(Talking, Giving/Taking\)](#)

[7.4 Interacting with Things that are Read \(Writing and Books\)](#)

[7.5 Removing Things from Play](#)

[7.6 Anticipating and responding to player commands](#)

Boyd, *An Introductory Guide to Inform 7*

[8. Times, Turns and Scores](#)

[9. Starting and Ending the Story](#)

[10. Creating New Kinds, Actions/Behaviours](#)

[11. Testing and Publishing](#)

Preface

This guide is intended to offer a distillation of the *Writing with Inform* (WWI) manual, focusing on those features that a novice interactive fiction writer would need to know and most likely want to know, organized in fashion that is most sensible and helpful to such a writer. This guide does not supplant WWI, and it refers to and directs the reader to WWI constantly, so it is best used in conjunction with WWI.

To understand what Inform 7 is, it might be helpful to read WWI 2.16, “Does Inform really understand English?”

1. The Inform IDE

To be completed. The “Source”, “Story” and “Documentation” tabs/windows.

1.2 The .inform and .materials file

When a new Inform project is created and saved, a file with an .inform extension will result. Depending on the platform being used, a file with a .materials extension will also be created (if not, it is created when we press the ‘Release’ button). If we look at this file in a file directory, we will initially only see a subfolder named “Release,” which will be empty. For more on this .materials file, see Section 11.

2. Preliminaries

2.1 Title and Byline; Division Headings

When we create a New Project, the name we give the file will be replicated as the title of our IF. This can be edited in the source text if we want a title other than the filename. The author byline will also be automatically generated from the owner of the computer the Inform program is installed on. This can also be edited in the source text.

Boyd, *An Introductory Guide to Inform 7*

For headings, see WWI 2.5 “Headings” & 2.6 “Why using headings is a good idea”.

In the “Source” window, at the top, there is a “Contents” and a “Source” tab. The “Contents” tab displays a schematic of the headings you are using. Inform has a hierarchy of five headings which are, from highest to lowest level: Volumes, Books, Parts, Chapters, Sections. We do not need to start with Volumes or use all of these headings; we could only use Chapters and Sections if we wanted to, for example. But we could not use Chapters as a subsection of Sections, for example, as that would not follow the hierarchy.

2.2 Including Extensions and Use Options

The built-in extensions that are currently included in our IF are marked with a black dot in the list accessible via the “Extensions” tab. By default, three extensions are automatically included when a New Project is created: Standard Rules and English Language by Graham Nelson and Locksmith by Emily Short. We can include other built-in options by declaring this at the start of our IF, e.g.,

Include Basic Screen Effects by Emily Short.

Use options (there are nine in total), which are also declared at the start of an IF, cover things like scoring, the display of room descriptions, the use of American dialect (i.e., spelling), and the serial comma, e.g.,

Use scoring.

For more details, see WWI 2.11 “Including Extensions” & 2.12 “Use options”.

2.3 A note about organizing an Inform work

While there is no required way to organize an Inform source text, a convention is to place the “Release along with a...” information (see Section 11) and the Included Extensions and Use Options (see section 2.1) in an opening section (with or after the Bibliographic information: see Section 11).

One possible way to divide up and organize the source code is to put all non-built-in modifications in one division (see Section 10), followed by a division containing the information about the world (spaces and objects; see Sections 3 to 6), and then a division about the plot containing how the player is able to interact with the world (see Sections 7 & 8).

3. Creating Rooms, Connections, and Things

Boyd, *An Introductory Guide to Inform 7*

Drawing from: WWI 2.1 “Creating the world”, 3.2 “Rooms and the map”, 3.3 “One-way connections”, 3.4 “Regions and the index map”, 3.5 “Kinds”, 3.18 “Articles and proper names”, 4.14 “Duplicates”, 13.3 “What are relations?”, 13.8 “The built-in verbs and their meanings”

Creating an IF typically consists of building (with words) a virtual environment that the reader-player moves in and interacts with (for brevity, we will use ‘player’ to refer to the reader-player). At the most fundamental level of Inform, this building is done by creating objects designated by nouns (of the IF author’s devising, except for directions) that are classified and understood by Inform as belonging to one of four object ‘kinds’:

- **room** (a space, not necessarily an interior one, that can be moved into and out of [from/to other rooms] by means of directions, and that can contain things);
- **region**, a space that can contain a collection of rooms (e.g., the separate spaces on the ground floor of a hotel). Regions can also contain (nest within) other regions;
- **direction** (Inform by default recognizes the eight cardinal compass points as well as ‘above’, ‘below’, ‘inside from’ and ‘outside from’);
- **thing** (an object within a room).

Inform has a number of special *things*:

- **container** (a thing that can have another thing *in* it and that can also potentially be entered/exited by a character)
 - **vehicle** (an enterable container that can moved by the player while in it)
- **supporter** (a thing that can have another thing or character *on* it)
- **backdrop** (a thing that can be present in multiple rooms and regions)
- **door** (a barrier/conduit between rooms)
- **device** (a thing that can be switched on or off)
- **person** (and **animal**)

In hierarchical terms, a special thing like a door is a *kind* of thing and a thing is a *kind* of object. A vehicle is a *kind* of container and a container is a *kind* of thing and a thing is a *kind* of object.

With these object kinds, one can write simple noun-verb-noun sentences in the present tense, called assertions, that declare the kind to which an object belongs and the relation between objects:

The Foyer is a room.

The Lounge is a room. The Lounge is north of the Foyer.

In the Lounge is a supporter called the table. A container called the vase is on the table. The flower is a thing. The flower is in the vase.

Boyd, *An Introductory Guide to Inform 7*

(One could also add, as a starting line:

The Ground Floor is a region. The Foyer and Lounge are in the Ground Floor.)

The first sentence of the above example declares that a noun, “Foyer” is a *room* object by using the equality *relation* (“is a”). Other assertion verbs define other built-in relations, such as adjacency (“is north of”), support (“is on”), and containment (“is in”).

The phrasing of the example above is rather long-winded and monotonous, although sometimes this degree of explicitness might be required if Inform gets confused. However, Inform *can* make certain inferences about assertions, allowing for a degree of flexibility and concision in how these assertions can be written. One could write the above example like this:

The Foyer is a room.

North of the Foyer is the Lounge.

A table is in the Lounge. On it is a vase. A flower is in the vase.

As this example shows, Inform can understand explicit assertions like the “is a room” in the first line, but can also (within limits) make inferences about which ‘kind’ a given noun belongs to without these explicit assertions. In the second line, it can infer that the Lounge is a *room* because it assumes that a noun (Lounge) that is associated with a room (Foyer) using a *direction* (North) is also a room. Because it understands that the Lounge is a room, it understands that anything that is stated as being “in” it (the table) is a *thing*, and further understands that the table is a *supporter* because there is something “on it” (the vase). It also can understand that the vase is a *container* because there is a thing “in” it (the flower), and, because of this chain of inferences, Inform does not mistake, for example, the vase as a *room* with a flower in it. But Inform’s ability to infer what kind a noun is has limits: for example, if there was nothing stated as being *in* the vase (e.g., if “A flower is in the vase” was omitted), Inform would not classify the vase as a container, only as a thing. If one wanted the player to be able to put something in a vase that was initially empty, one would have to add:

The vase is a container.

Or one could revise the second sentence in the third line (above) to read:

On it is a container called the vase.

Explicitly declaring a noun as the name of a kind: The example immediately above (“On it is a container called the vase.”) is an example of making the individual names of objects explicit, by using “called the”. This can be useful if Inform is misunderstanding or not recognizing a name for a kind (see 3.2).

Boyd, *An Introductory Guide to Inform 7*

Pluralizing & creating multiples of things: if we want a group of things (treated as a single unit), this can be accomplished with the phrase “Some”:

North of the Foyer is the Lounge. A table is in the Lounge. On it is a vase. Some flowers are in the vase.

We can make multiples of a thing by specifying a number:

Three flowers are in the vase.

For a technical description of Inform sentences, see the *Inform Recipe Book*, Example 223 “Formal syntax of sentences” (also available via WWI 13.2 “What sentences are made up from”).

3.1 Things in Parts (Composite Things)

Drawing on: WWI 3.23 “Parts of things”

Because in Inform a thing can belong to only one kind, the phrase “part of/parts of” can be used to combine, e.g., a supporter and a container (such as a desk with openable drawers), or a vehicle and a device, or a thing and a container and a device, etc.

A small table is in the Telephone Box. On the small table is a device called the telephone. The mouthpiece, the earpiece, and the receiver are parts of the telephone.

See also: WWI 4.15 “Assemblies and body parts”, 6.9 “Which and who”, 6.10 “Existence and there”, 6.11 “A word about in”, 6.12 “A word about nothing”

4. Describing Rooms and Things: Authored and Generated Descriptions

4.1 Descriptions of Rooms

Drawing from: WWI 3.10 “Properties holding text”

Assertions, like those given in section 3, pertain to those things that a player can interact with in some way. They are the *program* that is executed by the interpreter and are not the same as the text that a player will be reading. If someone was to play “Hotel Beaumonde” in the version given in section 3, they would be presented with the name of the starting room (by default the first room described), “Foyer” (in bold), and the input prompt:

Foyer

>_

If the player typed “look” (a common IF command) all they would get is the above output again. This is because all we have indicated about the Foyer is that it “is a room,” and the interpreter’s response to such an assertion is to display the name of the room in bold (to indicate to the player where they are). The player will not be told anything about the Foyer (there is nothing at this point it can tell), nor will they know that there is the Lounge to the north. This is where descriptions written by the author and placed within double quotation marks come in:

The Foyer is a room. "The Hotel Beaumonde's foyer has an old-world splendour that reminds you of classic films. The lounge is to the north."

The above example contains a brief description of the Foyer. (One could also write more explicitly: “The description of the Foyer is “The Hotel Beaumonde’s...”” or “The description is “The Hotel Beaumonde’s...””—the effect in terms of what the interpreter outputs is the same). Inform recognizes text within double quotation marks as descriptions and provides them for the player to read when the IF is run. This description is presented whenever the player is present in the Foyer. Descriptions like this are useful not only for enriching the player’s sense of the space they find themselves in, but also for alerting the player what they can do (and type into the prompt): in this example, the player is alerted to one thing (indeed the only thing) they can do—go “n” (“north”) to the Lounge.

The interpreter outputs the above example as:

Foyer

The Hotel Beaumonde's foyer has an old-world splendour that reminds you of classic films. The lounge is to the north.

> _

If we want the description of a room to display automatically only once (the first time the player enters a room), we can add “Use brief room descriptions.” between the title of the IF and the IF text proper (see 2.2). Even with this setting, the player is still able to display a room’s description by typing “look.”

See section 4.4 for providing alternate descriptions of a room after it has already been visited by the player or to reflect some changed condition.

4.2 Descriptions of Things

Drawing from: WWI 3.11 “Two descriptions of things”

Unlike rooms, things (including special things) can have two descriptions: one authored by the creator, the other generated by the interpreter.

Boyd, *An Introductory Guide to Inform 7*

Authored descriptions of things can be provided in the same way as is done for rooms (see 4.1 above). These will appear in the same way as room descriptions appear:

A table is in the Lounge. "In the centre of the lounge is a large, round, ornately-carved table with an elaborately veneered surface. A blue and white vase sits at its centre, holding a single lily."

However, if one does not want these authored descriptions of things to automatically appear when entering a room (and one doesn't usually want to provide a detailed description of every thing in a room at once), one can use "The description is..." or "The description of the X is...". This will result in the description only displaying if the player 'examines' ('x') the thing, supporter or container:

A table is in the Lounge. The description of the table is "In the centre of the lounge is a large, round, ornately-carved table with an elaborately veneered surface. A blue and white vase sits at its centre, holding a single flower." On it is a vase. The description of the vase is "A beautiful example of Japanesque Minton china, if you are not mistaken. A single flower sits in the vase." A flower is in the vase. The description of the flower is "A magnificent Calla lily."

In the example story being used so far, the player, when they go north, finds themselves in the "Lounge" and is informed (after the description of the Lounge):

You can see a table (on which is a vase (in which is a flower)) here.

Note that *this* description (or *response*) was not authored by the creator but was generated by the interpreter. By default, the interpreter creates a sentence consisting of a standard phrase ("You can see a...here.") and a list of things in the room with indications of their interrelations ("table (on which is a vase (in which is a flower))").

To modify the appearance of this generated description, see the next section.

4.3 Scenery, Backdrops, and Undescribed Things

Drawing from: WWI 3.8 "Scenery", 3.9 "Backdrops"

At times, one might want to describe things in authored descriptions of rooms and not have them also appear in the description of things that the interpreter generates, or may not want a player to 'see' some thing until they have examined some other thing in a room. Additionally, in a room that has multiple things, it might not be desirable to have all of these listed when (re)entering a room.

Boyd, *An Introductory Guide to Inform 7*

One way to remove things from the interpreter-generated description is to give them the property **scenery** (more on *properties* in the next section). Recall in the example in 4.2 that the interpreter generated the following awkward sentence:

You can see a table (on which is a vase (in which is a flower)) here.

If the table is included in the room description, i.e.,

The Lounge is north of the Foyer. "The Lounge is beautifully and luxuriantly furnished, with comfortable seating grouped in intimate clusters. A large table dominates the centre of the room."

then one might want to make the table scenery:

A table is in the Lounge. The table is scenery. The description of the table is "In the centre of the lounge is a large, round, ornately-carved table with an elaborately veneered surface. A blue and white vase sits at its centre, holding a single flower." On it is a vase. A flower is in the vase.

This will be displayed by the interpreter as:

The Lounge is beautifully and luxuriantly furnished, with comfortable seating grouped in intimate clusters. A large table dominates the centre of the room.

On the table is a vase (in which is a flower).

While the table is still mentioned, this is only because it is a *supporter* of a thing (the vase): note the difference between

You can see a table (on which is a vase (in which is a flower)) here.

and

On the table is a vase (in which is a flower).

The player can then 'examine' the table and be presented with the table description ("In the centre of the lounge is a large, round, ornately-carved table with an elaborately veneered surface. A blue and white vase sits at its centre, holding a single lily.").

But perhaps one doesn't want the player to be told about the vase and flower on the table until they examine the table, in which case the vase can also be made scenery:

Boyd, *An Introductory Guide to Inform 7*

A table is in the Lounge. The table is scenery. The description of the table is "In the centre of the lounge is a large, round, ornately-carved table with an elaborately veneered surface. A blue and white vase sits at its centre, holding a single flower." On it is a vase. The vase is scenery. A flower is in the vase.

The above will eliminate the generated description entirely (as the flower is in the vase, it will not be described either).

A **backdrop** is a particular type of scenery (in Inform terms, it is a *thing*, whereas *scenery* is a *property* of a thing) that can be present in multiple rooms or in a *region* (if a background is specified for a region, it will apply to all rooms in that region). One can specify which rooms a backdrop is in, or one can designate it as "everywhere":

The floor is a backdrop. It is everywhere. The description is "The marbled floor here gleams."

A player who examines the floor in the Foyer *and/or* in the Lounge will get the description above.

Scenery and *backdrop* are suitable for controlling the description of things that the author does **not** want or need to be taken or carried or moved (by the player or another agent): this is because they have the property *fixed in place*, which cannot be modified. To control the description of things that are to be taken or carried or moved, the property *undescribed* can be used to remove them from generated descriptions (usually desirable in cases where things are described elsewhere or where things are implicitly understood to be present):

A table is in the Lounge. The table is scenery. The description of the table is "In the centre of the lounge is a large, round, ornately-carved table with an elaborately veneered surface. A blue and white vase sits at its centre, holding a single flower." On it is a vase. The description of the vase is "A beautiful example of Japanesque Minton china, if you are not mistaken. A single flower sits in the vase." A flower is in the vase. The flower is undescribed. The description of the flower is "A magnificent Calla lily."

With the above, the interpreter will output:

On the table is a vase.

When the player examines the table and/or the vase, the descriptions of these will alert them to the presence of the flower, which they can then examine and/or take.

If a *supporter* (for example, the table in the above example) is given the *undescribed* property, nothing on top of it will appear in the generated description.

4.4 Varying Descriptions

Drawing from: WWI 3.14 “Devices and descriptions”, 5.6 “Text with variations”

By default, a room’s description will always be displayed when a player enters and re-enters the room. We can prevent a room’s description from displaying when a player re-enters a room by including “Use brief room descriptions.” in the prefatory section of our work (see section 2.2). Even with this setting the player will be able to access a room’s description with the command “look”. If we want a more granular control over how room descriptions display (on an individual room level), we can use, when writing rules to respond to player input (see section 7), the phrases “...without printing a room description.” or “...printing an abbreviated room description.” (both of which will only print a full description when the player visits a room for the first time).

We will be exploring Inform phrases and their use in creating rules in subsequent sections, but one use of these phrases that is pertinent here is for text substitution. In this use, phrases are contained within square brackets ([...]) and placed *within* the quotation marks of the description, before the text to which they pertain, as seen in the example below.

[if (condition)] / [unless (condition)]: This will check to see if the specified condition, in the first instance, is true, and in the second instance, is false, and output the text that follows. A common use for this is to change the description of a room depending on whether a player has already visited it or not, or to change the description of a thing depending on its state. Typically, you will want to have some alternate description if the condition evaluates as, respectively, false and true—this text can be placed after *[otherwise]* or *[else]*:

The Foyer is a room. "[if unvisited]The Hotel Beaumonde's foyer has an ostentatious splendour that makes it seem as if it were a set from [italic type]Die Fledermaus[roman type] or [italic type]La Traviata[roman type]. A constant stream of guests and staff pass in and out. There seems to be nobody loitering around; everyone appears to be [italic type]en route[roman type] somewhere. The lounge is to the north. [otherwise]Nothing has changed in the foyer since you were last here."

One can also use *[otherwise if]/[otherwise]* or *[else if]/[else]* to check multiple conditions of multiple things.

If you have brief room descriptions enabled, the alternate description will of course only display if the player enters ‘look.’ Otherwise no description at all will be displayed when a player re-enters a room.

4.5 Sounds and Smells, Taste and Touch

Drawing from: WWI 6.13 “To be able to see and touch”, 7.7 “The other four senses”

As has already been seen, a primary sense the player uses in IF is sight (looking, examining). We can specify if a thing is *visible* or *invisible* to the player (if a room is *dark*, things in it will be

Boyd, *An Introductory Guide to Inform 7*

invisible by default). The visibility or invisibility of things to the player or to other non-player characters can also be effected using the phrases “which can/cannot be seen by” and “can/cannot see”:

If the player can be seen by the counter-agents

If the counter-agents can see the player

We can also provide feedback if the player tries to use their other four senses:

listening to something

smelling something

tasting something

touching something

Instead of listening to the Lounge, say "The faint murmur of voices and the occasional clink of glassware floats in from the Bar."

Instead of smelling the lily, say "It has a delicate, calming perfume."

Instead of tasting the cut cigar:

if the cut cigar is switched on:

say "The cigar smoke has a distinctive floral, cedary taste.";

if the cut cigar is switched off:

say "This is a cigar, not chewing tobacco!"

See section 7 for an explanation of the “Instead of...” and “if” phrases and their use for creating rules.

See also: 2.3 “Punctuation”, 3.1 “Descriptions”, 5.2 “How Inform reads quoted text”, 5.8 “Line breaks and paragraph breaks”, 5.9 “Text with type styles”, 5.10 “Accented letters”, 5.11 “Unicode characters”, 5.12 “Displaying quotations”

5. Setting the States of Rooms and Things: Properties

WWI 3.6 “Either/or properties”, 3.7 “Properties depend on kind”, 4.6 “Properties again”

At this point, we have learned to create spaces (rooms and regions), position spaces in relation to each other with directions, place objects (things, supporters and containers) within rooms and add descriptions of rooms and other things for the player to read. We have also been introduced to the idea of objects having properties: so far, we have encountered *scenery* and *undescribed*, which modify how things can be revealed to a player (via generated descriptions). This section explores properties in more detail.

5.1 Spaces

Boyd, *An Introductory Guide to Inform 7*

Drawing from: WWI 3.15 “Light and darkness”

Rooms (and regions) have one property: *dark* or *lighted*. (This is an example of an *either/or property*: although there are two terms, these terms designate the possible states of a single property, i.e., ‘lightedness.’) Rooms are *lighted* by default. In a dark room, the player can still go in various directions, but cannot see the room description or interact with any of the objects in the room (except those being held).

The Telephone Box is a dark room.

A room’s darkness can be overridden if the player carries a thing with the property ‘lit’:

A device called the torch lighter is on the table. The torch lighter is lit.

(Other ways in which the dark/lighted and other properties can be changed by the player are dealt with in sections ??)

See also: WWI 12.19 “Changing visibility”

5.2 Things, Supporters, Containers

Drawing from: WWI 3.19 “Carrying capacity”, 3.22 “Food”

Things can be *edible* or *inedible*. By default, things are *inedible*. If a player (or something else) eats a thing, it will cease to exist in the world of the story.

By default, things and containers are *portable*: they can be ‘taken’ and ‘dropped’ or ‘put’ on something by a player or another agent (unless they have been made scenery or a backdrop). Things and containers can also be made to be *pushable*, which means they can be pushed around, but not carried/taken. Supporters by default are not portable (or pushable); they are *fixed in place*.

These defaults can be overridden:

The table is portable.

An ashtray is on the table. It is fixed in place.

Besides being portable, containers have a number of other either/or properties:

- open/closed
- openable/unopenable (*or* not openable)
- transparent/opaque
- portable/fixed in place
- lockable/unlockable (see next section)

Boyd, *An Introductory Guide to Inform 7*

- unlocked/locked (see next section)
- enterable (if the player or another agent can enter the container)

As can be seen from the example in the previous section (i.e., the vase, and how a player can interact with it), containers are by default open, unopenable, transparent, and portable (as well as unlockable and unlocked): thus, the player can see the flower in the vase. This default can be modified with the above properties, for example:

A container called a cigar box is on the table. The cigar box is closed, openable, and opaque. Some cigars are in the cigar box.

The maximum number of things that can be in a container or on a supporter (the default is 100) can be specified with *carrying capacity*:

The carrying capacity of the table is 5.

or

The table has carrying capacity 5.

5.3 Vehicles

Drawing from: WWI 3.16 “Vehicles and pushable things”

A *vehicle* is an enterable container that the player can move using directions (as if they were walking). As some vehicles are ridden by getting *on* them (tractors, bikes, horses, broomsticks) rather than *in* them, works that include these types of vehicle should include the following extension at the start of their work (see section 2.2):

Include Rideable Vehicles by Graham Nelson.

This enables the use of two more special things: *rideable vehicle* and *rideable animal*.

5.4 Doors, Locks and Keys

Drawing from: WWI 3.12 “Doors”, 3.13 “Locks and keys”

By default, Inform assumes that a player can freely move between rooms that are connected by directions. A *door* is a special thing for conduits between rooms that can take on properties that can be used to complicate this movement. A *door* does not have to be a literal door: any sort of connection between rooms (such as stairs or paths or windows) can be designated a *door*. Doors have the same properties as containers, as described in 5.2 above (except the *enterable* property):

The folding door is a door. It is closed. The folding door is inside from the Lounge. The folding door is scenery. The description is "A folding door made of small bevelled glass panes in a dark wood frame."

Like containers, doors can be locked/unlocked and lockable/unlockable, and the first property requires a *matching key* (a portable thing which can allow the player or another agent to change the lockable property from one state to the other):

The gold card unlocks the VIP door.

or

The VIP door has matching key the gold card.

Note that in the first example, Inform infers the gold card is a *matching key* because the instruction indicates it unlocks a door. A single key can be matched with multiple doors, if desired.

By default, a door is fixed in place, closed, openable, unlocked, unlockable, and without a matching key.

To create a one-sided door (one that is only visible from one of the rooms it connects, and which can only be gone through in one direction), use “through the”:

The fire ladder is a door. It is north of the Suite. Through the fire ladder is the Laneway.

5.5 Devices

Drawing from: WWI 3.14 “Devices and descriptions”

A *device* is a thing (usually a mechanical object like a light switch or a radio) that can be either *switched on* or *switched off*. Devices are by default *switched off*.

A small table is in the Telephone Box. On the small table is a device called the telephone.

6. Living Creatures

Drawing from: WWI 3.17 “Men, women, and animals”, 3.19 “Carrying capacity”, 3.20 “Possessions and clothing”, 13.4 “To carry, to wear, to have”

Human characters can be designated *person*, *man* or *woman*.

The waiter is a person in the Lounge.

The waiter is a man in the Lounge.

The waiter is a woman in the Lounge.

The relevance of gender here is primarily about how Inform/the interpreter understands the player’s use of “him” or “her”: if there is a male character and a female character in the same room at the same time, the interpreter will know who is being referred to when the player uses

Boyd, *An Introductory Guide to Inform 7*

“him” or “her.” By default, a *person* is male, so in the first example above, if there was no character defined as a female in the room and the player typed “look at her,” the interpreter would reply: “I’m not sure what ‘her’ refers to.”

An *animal* is also by default male, but can be designated a *female animal*, or *neuter animal*. A neuter animal will be referred to as “it.”

The fly is a neuter animal in the Lounge.

Living creatures, including the player character, can *carry* and *wear* things:

The waiter carries a silver tray.

or

The silver tray is carried by the waiter.

The waiter wears a white suit jacket.

or

The white suit jacket is worn by the waiter.

If a character is *carrying* a wearable item, we have to specify that the item is *wearable* (by default, it would be considered a generic *thing*). The above examples use the carrying and wearing relations; other relations that can be used to assign objects to characters are the possession relation (“has”) and the concealment relation (“conceals”).

Creatures, including the player character, like containers and supporters, can be given a maximum *carrying capacity*:

The carrying capacity of the waiter is 2.

6.1 Movement of non-player characters/creatures

Drawing from: WWI 6.14 “Adjacent rooms and routes through the map”

To randomly move a NPC between a series of rooms, see section 8, “Times, Turns, and Scores.”

7. Responding to Player Input

Drawing from: WWI 2.2 “Making rules”, 7.2 “Instead Rules”, 7.3 “Before Rules”, 7.5 “After Rules”, 7.8 “Rules applying to more than one action” 7.9 All actions and exceptional actions”, 7.10 “The noun and second noun”, 7.12 “In the presence of, and when”, 8.11 “Now...”, 8.13 “Checking on whereabouts”

All the previous sections focus on things we can do in order to build the *initial state* of the story world. But we also have to manage what happens when the player interacts with the story world by entering commands to take specific actions. Inform has a range of built-in, standard

actions that it recognizes (see Index-->Actions Index) and default responses (or *reports*) for these standard actions. These default responses can be very useful: for example, any attempt by the player to *attack* or *kill* other characters or creatures is by default disallowed, and generates the default report, "Violence isn't the answer to this one." We, in many cases, will want to modify these default responses and messages.

In a manner analogous to things and their properties, we can manage how our IF responds to actions with *rules*. Section 7.1 looks at three of the most used rules:

- **Before** rules, which enable preliminary activities to happen before the action is tried;
- **Instead** rules, which block or divert the action, possibly causing something entirely different to happen than that expected by the player;
- **After** rules, which enable actions or responses after the action has taken place.

7.1 Basic Rules (Instead, Before, After; If, Now)

We have already seen that we can manage how the player can (initially) interact with the story world through properties (by retaining or modifying the default properties of things). For example, in "Hotel Beaumonde," we have indicated that the ashtray (a thing) is *fixed in place* (to override the default property of things as *portable*):

A container called a cigar box is on the table. The cigar box is closed, openable, and opaque. Some cigars are in the cigar box. An ashtray is on the table. It is fixed in place.

If the player tries to 'take' the ashtray, the interpreter will respond with a built-in report for things that have the property *fixed in place*: "That's fixed in place." This may puzzle the player, as ashtrays on tables are usually not immovable or fixed in place in the real world. If we don't want the player to be able to do something that they could reasonably do, but want to offer a reasonable explanation for why they can't (or shouldn't) do it, we can use the **Instead of..., say** rule:

A container called a cigar box is on the table. The cigar box is closed, openable, and opaque. Instead of taking the cigar box, say "Taking the box would demonstrate very bad manners, and you decide against it." Some cigars are in the cigar box. An ashtray is on the table. Instead of taking the ashtray, say "You have no need to encumber yourself with a heavy cut-glass ashtray."

Other common built-in player action verbs besides 'taking' (as in the example above) can also be managed, e.g.:

Instead of sitting on the throne, say "You'd better not—the King might catch you."
Instead of wearing the crown, say "It's not worth losing your head over."

Boyd, *An Introductory Guide to Inform 7*

The Instead rule only works in situations where a player can feasibly perform an action, e.g., take an object from a box that is unlocked (if the box was locked, then the option to take the object inside would not be available).

If we anticipate that a player will need to or likely want to perform or try a range of different actions in relation to a thing, we can list these together instead of writing separate rules:

Instead of attacking or dropping the vase

We can also manage what a player can do with a thing using “Instead of doing something with the” (which would, because of the use of “something”, prevent the player from performing *any* action on the thing in question) and “Instead of doing something other than [action(s)] with the” (note that the ‘with’ must be used):

Instead of doing something other than examining with the cigar band, say "It is enough to see the name on the band; there is no need to otherwise bother with it."

We might want these restrictions to depend upon whether a non-player character is in the same room as the player, or if some other condition is true; in these cases we can use the phrasing above, but add, respectively, “in the presence of” or “when [condition]”:

Instead of doing something in the presence of the counter-agents: say "The two men pin you between them and rush you out into the street, forcing you into a waiting car."

Instead of doing something when the waiter is carrying the gun:
say "You see the gun in the waiter's hand; the two men rush towards you!";
try going south.

We also might want to have a customized response to actions that the player might take to a class of things in general. For example, if the player tries to ‘eat’ things that are inedible (the default property for things), the interpreter will respond with the built-in phrase: “That’s plainly inedible.” This can be customized to output a response that includes the name of the thing that is the focus of the action:

Instead of eating something inedible, say "Eating [the noun] will give you indigestion, to say the least."

The phrase “[the noun]” is an example of a *token*: this one will output a statement with the name of the thing the player has tried to eat. For more on tokens or text substitutions, [see](#).

See *also*: 14.2 “Adaptive text” (object-verb agreement), 14.3 “More on adapting verbs”, 14.5 “Adapting text referring to other things [besides the player]”, 14.6 “Adapting demonstratives and possessives”; 14.8 “Adapting contractions”

Boyd, *An Introductory Guide to Inform 7*

Another common rule, **Before**, checks for a certain condition before responding to player input (often using one or a combination of **If**, **now**, **say**):

Before switching on the cut cigar:

if the player is holding the torch lighter, now the cut cigar is switched on;
say "You carefully apply the flame of the torch lighter until the end of the cigar
glows red."

If a player types a command like "light cigar," this example evaluates whether the player is in possession of the thing that is required to light the cigar (the torch lighter), and, if true, changes the state of the cut cigar from *switched off* to *switched on*. Note the use of the colon and semi-colon, which are required, as is the *tabbed* indentation. If this formatting is not followed, an error message will result when testing.

If we want something to happen before a player enters a room with a door, we should use the Before statement on the door, not the room. In our example story, let's say we want to remove the cigar from the player before they enter the Telephone Box, which has a closed, unlocked door (which by default will be automatically opened if the player tries to enter the Telephone Box). This will work (whereas it will not with "Before entering the Telephone Box:"):

Before opening the folding door:

if the player is carrying the cut cigar:
now the cut cigar is in the ashtray;
say "(first stubbing out your cigar in the ashtray on the table)"

See WWI 6.14 "Adjacent rooms and routes through the map" & 6.15 "All, each and every" for more sophisticated deployments involving multiple like objects.

"If" and "Now" are very useful and flexible terms to, respectively, check to see if a condition is true or false, and change the state of something (often after checking whether a condition is true or false). As seen in the example above "if" and "now" are often used together; however, another common use of "now" is in conjunction with "carry out [action]" which changes the state of something when an (common) action is taken:

A device called the light switch is part of the folding door. The light switch is switched off.

Carry out switching on the light switch: now the Telephone Box is lighted.

Carry out switching off the light switch: now the Telephone Box is dark.

See WWI 11.6 "If", 11.7 "Begin and end", 11.8 "Otherwise", 11.9 "While" for more about using "if" and associated phrases such as "unless", "begin", "end if", "otherwise"/"else", and "while"

Boyd, *An Introductory Guide to Inform 7*

to check for multiple conditions to determine a response. See WWI 8.13 “Checking on whereabouts”, for a sample list of common “if” commands.

The **After** rule can be used to customize the default responses from the interpreter, such as the “Taken.” message after the player’s character takes something. For example, if a player can and does eat something, the default message is: “You eat the [thing]. Not bad.” This can be changed with the *After* rule:

After eating the paper, say "You slowly chew the paper into a mush and swallow a couple of times. You try not to grimace."

Another way of managing and modifying default responses (or *reports*) is to use “Report...: say “...” instead. In this example, this prevents the rather odd-sounding default message (“You get into the ottoman.”) from appearing:

The ottoman is an enterable scenery supporter in the Telephone Box. Report entering the ottoman: say "You settle down on the comfortably stuffed ottoman." instead.

For a more sophisticated method for globally or dynamically changing default responses, see WWI 14.10 “Responses” and 14.11 “Changing the text of responses”.

Inform has around 400 built-in phrases like Instead, Before, After, If, Now, Say, Carry out, etc., to create rules like the above. Besides being used to create rules, phrases are used to determine if a condition is true or false, to decide a value, and to substitute text. For more on phrases, see WWI 11 “Phrases”.

For a helpful flowchart of how Inform processes Before, Instead, and After rules, see WWI 12.2 “How actions are processed”. As that section notes, every action is processed through three rules: the “check” rule (which determines whether an action is valid); the “carry out” rule (which performs the action); and the “report” rule (which tells the player what happened as a result of the action).

WWI 12.21 “Guidelines on how to write rules about actions” points out that Before, Instead, Check, Carry out, After and Report can often be used interchangeably, and recommends following established conventions when deciding which one to use.

See also: WWI 6.14 “All, each, and every”, 6.16 “Counting while comparing”, 7.11 “In rooms and regions”, 7.13 “Going from, going to”, 7.14 “Going by, going through, going with”, 7.15 “Kinds of action”, 7.16 “Repeated actions”, 7.17 “Actions on consecutive turns”, 12.16 “Reaching inside and reaching outside rules”, 12.18 “Changing reachability”

7.2 The Player Character (Placing, Appearance and Inventory)

Drawing from: WWI 3.19 “Carrying capacity”, 3.20 “Possessions and clothing”, 3.21 “The player’s holdall”

By default, the player starts in the first room that is described in the Inform source text. To place the player in a different room at the start, we would include the command, “The player is in the [desired room].” We can also move the player at will (say, in response to a particular action they take) to a room other than the one they are currently in with “move the player to [desired room]” (we could also use “now the player is in [desired room]”).

A common opening player command in IF is “x [examine] me”. The default response to this is, “As good-looking as ever.” To customize the description of the player character, use “Instead of examining the player”:

Instead of examining the player, say "Your outfit was chosen to give you the look of the kind of wealthy guest to whom the Hotel Beaumonde caters: not the wastrel or gadabout kind, but rather the professional, the person of business."

We can include in the description of the player (or another character) a list of what is being worn and carried, which is useful if we plan to have these items change possession or quality in the course of play and we expect the player to repeatedly check their current inventory:

You are wearing [a list of things worn by the player].
You are carrying [a list of things carried by the player].

The portions in square brackets are examples of tokens that will substitute a list of the things currently being worn or carried by the player for the bracketed phrase.

The other common opening player command in IF is “i [inventory]”. This will automatically list everything the player is carrying and wearing.

In some circumstances, a creator might want to limit what the player character can *carry* (say one item in each hand), but might want the player character to be able to transport a number of things that they can choose from to have in their hands: a special container called a *player's holdall* can be used for this (note that this has to be described as “a [not ‘the’] player's holdall”]):

The valise is a player's holdall. The player is carrying the valise. The valise is closed.

The player's carrying capacity is 2.

If the player's carrying capacity has already been reached, any additional things taken by the player will automatically be placed in the holdall.

7.3 Interacting with Non-Player Characters (Talking, Giving/Taking)

Drawing on: WWI 3.24 “Concealment”, 6.13 “To be able to see and touch”, 7.6 “Reading and talking”, 8.7 “Moving things”, 12.14 “Actions for any actor”

One of the common actions a player performs in regard to non-player characters is talking with them. The terse forms of commands that players can use to talk to non-player characters are usually one of these:

Ask [character] about [something]
 Tell [character] about [something]
 Answer [something]
 Answer [character], [something]
 [character], [something]

These exchanges make use of the “After” rule (mentioned at the start of Section 7), which checks for keyword(s) entered by the player, and, if true, provides a response:

After asking the waiter about "lily", say "He says 'Yes, sir, it's quite a beautiful example of the Madonna lily.'"

After telling the waiter about "Calla lily", say "He says, 'I feel quite certain, sir, that it is a Madonna lily.'"

After answering the waiter that "Calla lily", say "He says, as if he expected your response, 'Yes, sir--I am sure you are quite right. It *is* a Calla lily.' He pauses, and quickly scans the room."

As asking, telling, and answering are often used interchangeably, they can be combined:

After answering or telling the waiter about "Calla lily", say "He says, 'I feel quite certain, sir, that it is a Madonna lily.'"

Note that using *asking* and *telling* requires the use of “about” before the topic and using *answering* requires the use of “that” before the answer phrase, including for yes/no answers:

After answering the waiter that "yes":

If we are combining *answering*, *asking*, and *telling*, we need to be sure that the last term and the term preceding the topic/answer match: if *answering* is the third term, use “that”; if *asking* or *telling* is the third term, use “about”:

After telling or answering the waiter that "yes":

After answering or telling the waiter about "yes":

If we want the player to be able to converse with an NPC about a range of topics, we can use a *table* with a *topic* column that tries to match player input with a response from the NPC: see WWI 16.3 “Topic columns” (and 16.1 “Laying out tables”). We can also create a similar table

Boyd, *An Introductory Guide to Inform 7*

with a “item” column instead of a “topic” column if we want the player to be able to converse with an NPC about objects in the story (see Recipe Book example 297, “Nameless”).

Another common action we might want a non-player character to perform is to give (or allow the player to take) something that the non-player character is in possession of (by default, the player cannot take something in the possession of a non-player character), or allow a non-player character to take something, perhaps something in the possession of the player. This example allows the player to take something (the cut cigar) possessed by the waiter, but only if the player ‘asks’ for it:

Instead of asking the waiter for the cut cigar:

move the cut cigar to the player;
say "The waiter says, 'Certainly, sir. I've just cut it.' He holds out the tray. 'You're very kind to have cut it,' you say, taking it, but keeping your eyes on his face. He nods gravely, returning your gaze. His gaze then shifts to the table. 'There's a torch lighter here on the table, sir.'"

If we want a non-player character to accept something from the player (or vice-versa), we can specify what happens when something is given to the non-player character:

Instead of giving the flower to the waiter:

now the flower is in the vase;
say "The waiter accepts the lily and places it back in the vase."

We could also have written, as the second line above, “now the waiter carries the flower” (if we wanted the waiter to retain possession of the flower).

We can use the property of being ‘touchable’ to indicate that something (or a class of things) can be taken by the player or a non-playable character. In this example, if the gun is in the same room as the waiter, the waiter can take the gun:

If the waiter can touch the gun which is not carried by the waiter:

try the waiter taking the gun;
say "The waiter approaches the men, and suddenly stumbles. He falls against them, and a flurry of arms shoot out, as the trio brace each other to regain their balance."

If we want to ensure that the player’s possessions are not purloined, we could write “which is not carried by the player”.

Often, we will not want the player to be able by default to see what a non-player character is carrying. To effect this, we need to declare a rule about concealed possessions:

The counter-agents are in the Bar. The counter-agents carry the gun.

Rule for deciding the concealed possessions of the counter-agents: yes.

The ‘yes’ indicates that the possessions of the counter-agents *are* concealed from the player (see WWI 18.9 “Deciding the concealed possessions of something”).

When we write rules for actions that do not explicitly indicate who is performing them, the default assumption is the rules only apply to the player. If we want a rule to apply generally to the player and to other characters, we need to include the phrase “an actor”:

Instead of smelling the flower
Instead of an actor smelling the flower

For more on NPCs, see: WWI 12.13 “Giving instructions to other people”, 12.4 “Persuasion”, 12.5 “Unsuccessful attempts”, 12.6 “Spontaneous actions by other people”, 12.11 “Making actions work for other people”, 12.12 “Check rules for actions by other people”, 12.13 “Report rules for actions by other people”

7.4 Interacting with Things that are Read (Writing and Books)

Drawing from: WWI 7.6 “Reading and talking”

In cases where we have something like a letter (or note or sign, etc) that we want the player to be able to read, we can use the composite things method (“is part of”) we saw in Section 3.1:

Something called a paper is in the envelope. The paper is edible. The description of the paper is "A thin strip of flimsy paper, with a one-line typewritten message." The message is part of the paper. The description of the message is "The message reads: [italic type]Do not Count a single pair of lovers. Trust the kindest cut of all. Insist on Calla for the call."

The player can then “read message” (“examine”, “look at”, and “read” are treated as synonymous by Inform, so any of these commands would work). If the player tries to “read paper”, they will be given the description of the paper, with a prompt that there is a message that can be read.

As with conversations with non-player characters, there are a range of commands that Inform recognizes when interacting with books:

look up [text] in [something]
consult [something] on/about [text]
read about [text] in [something]
read [text] in [something]

Boyd, *An Introductory Guide to Inform 7*

The first two commands are used by players when they are trying to find some information in a reference work of some sort. All of these commands are responded to by the “consulting the [thing, usually a book] about [topic].

On the small table is a directory. After consulting the directory about "Covent Garden 1123", say "It's the number for a costumiers called 'Wopsle & Co.'"

In the same way as conversing with an NPC about a range of topics (discussed in the previous section), we can use a *table* with a *topic* column that tries to match player input with a passage from a book: see WWI 16.3 “Topic columns” (and 16.1 “Laying out tables”).

See Recipe Book example 94, “Costa Rican Ornithology,” for an example of a book using a table (also accessible via WWI 7.6). For another book example, see Recipe Book example 303, “Pages”. For an example of a bookshelf or library of books that can be examined, see Recipe Book example 324, “Bibliophilia”.

7.5 Removing Things from Play

Drawing from: WWI 8.10 “Removing things from play”

If we want to remove something from play (because it has been destroyed or otherwise rendered permanently inaccessible), we can indicate that it is now “nowhere”.

Instead of attacking or dropping the vase:

now the vase is nowhere;

say "The vase shatters on the marble floor. You feel a temporary hush in the room and eyes turning in your direction. The waiter gestures to a nearby bellhop, who quickly sweeps up the debris."

Note that, by default, a thing in a container (**or on a supporter?**) that is “nowhere” will also be automatically removed from play, unless otherwise provided for.

7.6 Anticipating and responding to player commands

Drawing from: WWI 17.2 “New commands for old grammar”, 17.11 “Understanding values”, 17.19 “Does the player mean...”, 17.21 “Understanding mistakes”

Inform recognizes many commands that players might type into the prompt (see Actions Index -> Commands) and so we don’t need to worry about the player not getting an understandable response in many instances. However, we might want to catch expected commands (often synonyms of built-in commands) that Inform does not automatically recognize and equate them with a built-in command. (When the interpreter does not understand a command, it will output “That’s not a verb I recognize.” if it pertains to an action or “You can’t see any such thing.” if it pertains to an object.) We can do so using “Understand”:

Boyd, *An Introductory Guide to Inform 7*

Understand "flip [something switched off]" as switching on. Understand "flip [something switched on]" as switching off. Understand "flip [something]" as switching on.
Understand "press [something switched off]" as switching on.

We can also use “Understand” to respond to synonyms of rooms/objects that the player might enter in the prompt:

A dark grey Chesterfield overcoat is worn by the player. Understand "coat" as the overcoat.

Multiple related synonyms can be included in an instruction: see WWI 17.12 “This/That”. We can also use “Understand” to recognize commands that refer to a property of an object: see WWI 17.15 “Understanding things by their properties”.

Another useful way of sensibly responding to player commands, in instances of potential confusion, is to use the “Does the player mean” instruction. In the example below, we want to player to obtain the cut cigar when entering the command “ask waiter for cigar”; however, it is not clear to the interpreter whether “cigar” in this command refers to the cut cigar, the cigar band, or the cigar box (so, when the player types “ask waiter for cigar”, they get the rather annoying response: “Which do you mean, the cigar box, the cigar band, or the cut cigar?”). We can specify what “cigar” refers to in this situation (i.e., asking the waiter):

Does the player mean asking the waiter for the cigar box: it is very unlikely.

Does the player mean asking the waiter for the cigar band: it is very unlikely.

Or (probably more simply),

Does the player mean asking the waiter for the cut cigar: it is very likely.

The appropriate values are: very likely, likely, possible, unlikely, very unlikely.

A third way of helpfully responding to player commands, especially ones that might indicate that the player needs assistance, is to use the “Understand...as a mistake” instruction:

Understand "telephone [text]" as a mistake ("Of course, you need to take the earpiece and listen for the operator, and then give them the number.") when the location is the Telephone Box.

For redefining or customizing built-in commands, see WWI 17.3 “Overriding existing commands” and 17.4 “Standard tokens of grammar”. For using “understand” with new actions, see section 10.

8. Times, Turns and Scores

Boyd, *An Introductory Guide to Inform 7*

Drawing from: WWI 9.2 “Awarding points”, 9.5 “Every turn”, 9.11 “Future events”, 9.13 “The past and perfect tenses”, 9.14 “How many times?”, 9.15 “How many turns?”

We may want to respond in a specific way to repeated visits to a particular room or respond to an input repeated by the player in connection to a particular thing or to a particular circumstance, perhaps to tell the player that repeating an action will have no (additional) outcome or to tell the player that they need to repeat an action to achieve an outcome:

Instead of answering, telling, or asking the waiter about "Calla lily" for the third time, say "The waiter merely raises his eyebrows inquisitively."

In the above example, “for the third time” could also be written “three times” or “3 times.” In this example, the response will only occur the third time the player tries to elicit a response from the waiter about the Calla lily, and we might want this response to be outputted every time the player performs this action the third time and additional times:

Instead of answering, telling, or asking the waiter about "Calla lily" for more than the third time, say "The waiter merely raises his eyebrows inquisitively."

We can also qualify times with “less than”, “at most”, “at least”, and “only” or “exactly”.

A **turn** represents a single input by the player and the interpreter’s response to that input. (If we are keeping track of time, a turn represents 1 minute: see WWI 9.6 “The time of day”.) We can track turns in the same way as we track times, above.

We may want something to occur every turn or to check every turn to see if some condition is true or false. For this, we can use “every turn”.

Every turn:

if the player has been in the Telephone Box for 20 turns:
move the counter-agents to the Lounge;
say "Through the panes of the door, you see two men in dark coats and bowlers enter the Lounge and slowly sweep the room with their eyes."

The above example checks for a specific number of turns. We can also use “every turn” to check if a specified number of *times* has been met. As can be seen in the above example, we can use the present tense (is) or the present perfect tense (has been) depending on our goal:

if the player is in the Telephone Box for the third time
if the player has been in the Telephone Box 3 times

The outcome connected to the first would occur when the player is in the Telephone Box the third time (and we wanted something to happen as a result); the second would likely be used in situations when the player was not in the Telephone Box but had been in it three times already,

Boyd, *An Introductory Guide to Inform 7*

for example, if we wanted to prevent the player from re-entering the Telephone Box for the fourth time.

“Every turn” is useful for continuously and randomly moving a NPC between a series of rooms that are adjacent to each other (and that have no obstacles between them, such as a locked door):

A bellhop is a man in the Lounge. The bellhop is undescribed.

Every turn:

if the bellhop is in a room (called the current space):
 let next space be a random room which is adjacent to the current space;
 if the bellhop is visible, say "A bellhop rushes into [the next space].";
 move the bellhop to next space;
 if the bellhop is visible, say "A bellhop rushes in from [the current space]."

As this example shows, we can check to see if the bellhop is “visible” to the player (that is, is in the same room as the player) before providing a description of the bellhop’s actions. The “let next space be a random room which is adjacent to the current space” line provides a glimpse of how complex Inform’s programming can get (e.g., “random” and “adjacent” are specific concepts recognized by Inform).

We can also use the phrase “every turn when [condition]”:

Every turn when the player is carrying the cut cigar:

if the cut cigar is switched on, say "You draw on the cigar and expel a puff of fragrant smoke."

In some cases, having something repeated on every turn can become tedious for the player. To deal with this we can randomize the number of turns that can pass before an action occurs. For example, in the two examples above, we can modify the “Every turn:” line to:

Every turn when a random chance of 1 in 2 succeeds:

Again, “random chance of 1 in 2 succeeds” shows Inform’s ability to complexify basic commands like “every turn”. For more details about randomness, see WWI 8.18 “Randomness” and 8.19 “Random choices of things”.

To use a **points/scoring system**, we must enable this feature of Inform by including “Use scoring.” in the prefatory section of our story (see 2.2). We can then, in response to player or other actions “increase the score by [number]” or “decrease the score by [number]” (or declare “now the score is [number]”).

9. Starting and Ending the Story

Drawing on: WWI 9.1 “When play begins”, 9.4 “When play ends”, IDE Extensions tab: Basic Screen Effects

Typically, an IF does not simply plop the player into a space without context (although some intentionally do!). Many IFs start with a preamble, before the title, author and release information (known as the “banner text”), first room, and input prompt is displayed. This is done using “When play begins:”

When play begins:

```
say "1938. London. The city's famously un navigable fogs seem inconsequential
compared to the dark cloud spreading over Europe and threatening to cross the
Channel. You have vital intelligence that you need to pass to an as yet unknown
contact...."
```

To have a preamble appear on a separate screen before the banner text and starting location, we need to include a built-in extension into our IF, by adding “Include Basic Screen Effects by Emily Short.” at the start of the work (see 2.2.). Then we can add to “When play begins:” something like:

When play begins:

```
say "1938. London. The city's famously un navigable fogs seem inconsequential
compared to the dark cloud spreading over Europe and threatening to cross the
Channel. You have vital intelligence that you need to pass to an as yet unknown
contact...
```

```
(Press SPACE to begin);
wait for the SPACE key;
clear the screen.
```

or

When play begins:

```
say "1938. London. The city's famously un navigable fogs seem inconsequential
compared to the dark cloud spreading over Europe and threatening to cross the
Channel. You have vital intelligence that you need to pass to an as yet unknown
contact...";
pause the game.
```

In the first example, we could have used “wait for any key” instead of “wait for the SPACE key”; the “clear the screen” phrase clears the preamble before displaying the banner text and the starting room. In the second example, the phrase “pause the game” generates “Please press

Boyd, *An Introductory Guide to Inform 7*

SPACE to continue.” when the work is run (note that “pause the game” can be used anywhere in a work, not just at the beginning, if we want to include a ‘break’).

The Basic Screen Effects extension also has a nice feature if we want to use an epigraph:

```
When play begins:
    display the boxed quotation
    "How oft when men are at the point of death
    Have they been merry! -- Romeo and Juliet";
    show the current quotation;
    pause the game.
```

A boxed quotation can be incorporated with a preamble:

```
When play begins:
    display the boxed quotation
    "How oft when men are at the point of death
    Have they been merry! -- Romeo and Juliet";
    show the current quotation;
    say "1938. London. The city's famously unnavigable fogs seem inconsequential
    compared to the dark cloud spreading over Europe and threatening to cross the
    Channel. You have vital intelligence that you need to pass to an as yet unknown
    contact...";
    pause the game.
```

An IF can have one or multiple endings, which are usually the outcomes of specific conditions. We can indicate the end of the story using the phrases “end the story” or “end the story finally” (which will print out “The End”) or “end the story saying” or “end the story finally saying” which can be used to print a customized message.

We can also use the “when play ends” phrase with “resume the story” if we want to give the player the chance to continue playing rather than ending the story. We can use the “if player consents” phrase to ask the player to type “y” (yes) or “n” (no):

```
The Street is south of the Foyer.
```

```
Instead of going south from the Foyer:
    if the score is 9, end the story finally saying "You leave the Hotel Beaumonde, your
    mission completed.";
    if the score is less than 9, end the story.
```

```
When play ends when the story has not ended finally:
    say "You haven't completed your mission yet.
```

```

Do you want to continue? >;
if the player consents:
    resume the story;
    try looking.

```

10. Creating New Kinds, Actions/Behaviours

Drawing from: 4.1 “New kinds”, 4.2 “Using new kinds”, 4.3 “Degrees of certainty”, 4.4 “Plural assertions”, 4.5 “Kinds of value”, 12.7 “New Actions”, 12.8 “Irregular Verbs”, 12.9 “Check, carry out, report”, 12.15 “Out of world actions”, 12.17 “Visible vs touchable vs carried”; 13.9 “New assertion verbs”; 13.10 “Defining new propositions”, 13.12 “Relations which express conditions”; 17.1 “Understand”, 17.13 “New tokens”

We have seen how we can assign values to built-in kinds, e.g., “The vase is a container.” (“vase” is the value, and a container is a kind of thing, and a thing is a kind of object, so vase > kind of container > kind of thing > kind of object). Other top level kinds besides object are text, number and time. If we wanted to make a new *kind* of container (not just a new container with a specific name), so that we could differentiate some containers from others by kind of container, we could create a new kind and associate it with the container kind, using “is a kind of”:

```

A fluid container is a kind of container.
A drinking vessel is a kind of fluid container.
A jug is a kind of fluid container.

```

Then we can create different drinking vessels and jugs:

```

A skyphos is a drinking vessel.
A kylix is a drinking vessel.
A krater is a jug.
A hydria is a jug.
An oinochoe is a jug.

```

We could then write rules that would, for example, only allow the player to drink out of something that was a drinking vessel, or that would only allow jugs to have water or wine poured into/from them.

We can write the above examples more concisely as:

```

Drinking vessel and jug are kinds of fluid container.
Skyphos and kylix are kinds of drinking vessel.
Krater, hydria, and oinochoe are kinds of jug.

```

Boyd, *An Introductory Guide to Inform 7*

As we have seen, kinds have properties, and we can create new properties, both either/or properties and value properties:

A couch is either occupied or unoccupied.

or

A couch can be occupied. [in this example, the 'or' value is silently created: "not occupied".]

To create value properties, we use the phrase "has some text called the"

The Andron has some text called the music. The music of the Andron is usually "a muted strumming from a lute".

In some cases we want to create values for things that are concepts rather than objects: concepts like, e.g., potency or liquid volume. We can do this using "kind of value". Kinds of values can be qualitative or quantitative:

Potency is a kind of value. The potencies are unmixed, strong, well-mixed, and weak.
Measure is a kind of value. 1 ladleful specifies a measure.

We can then assign these values to objects:

The krater has a measure of 30 ladlefuls.

We can also create new values that are part of a 'condition' property.:

A drinking vessel can be full, partly drained, or empty, A drinking vessel is usually empty.
A jug can be full, partly drained, or empty. A jug is usually full.

In the above examples, a property is silently created called "drinking vessel condition" and "jug condition."

The "usually" allows us specify a different default or starting property for a specific instance (in this case of a particular drinking vessel or jug):

A krater is usually empty.

Besides "usually", we can also use "seldom", "always" and "never".

Kinds: object, text, number, time

Boyd, *An Introductory Guide to Inform 7*

Object kinds: room, thing
 “kind of value”

To create new directions, see WWI 3.26 “Directions.”

11. Testing and Publishing

11.1 Testing

Drawing from: 1.4 “The Go! button”, 1.5 “The Replay button”, 1.6 “The Index and Results Panels”, 1.7-1.9; 2.4 “Problems”, 2.7 “The SHOWME command”, 2.8 “The TEST command”, 2.9 “Material not for release”, 7.1 “Actions”, 14.2 “The RESPONSES testing command”

Testing procedures for Inform are fairly straightforward.

A suggested method for testing is to have a Test list for each section of your IF (at the end of the section), and a “Test all” for a full test at the start of the IF.

11.2 Publishing

Drawing from: WWI 25.2 “Bibliographic data”, 25.6 “The Release button and the Materials folder”, 25.8 “Cover art”, 25.9 “An introductory booklet and postcard”, 25.10 “A website”, 25.11 “A playable web page”; 25.16 “Walkthrough solutions”; 25.17 “Releasing the source text”

Besides providing the title and author of our IF, we might want to provide some additional information about the work in a standardized way. The title and byline that are generated when a New Project is created are an abbreviated form of:

The story title is “[Title]”.
 The story author is “[Author]”.

Similar declarations can be used for story headline (i.e., subtitle), story creation year, story genre, story description, and release number. The values for all of these, except for year and release number, are placed in quotation marks. These are optional, but Inform will provide default values for title, author, headline, genre and release number if these are not specified. For genres, the suggested options are provided in WWI 25.3 “Genres”.

While not necessary, it is good practice to provide cover art for an IF. For details about how to prepare a cover, see WWI 25.8 “Cover art”. The cover art files are included in the .materials file/folder (not in the ‘Release’ subfolder).

In addition to *cover art* we can include in our release a prepared *introductory booklet* about playing IF (by Emily Short) and/or an *introductory postcard* about IF (by Andrew Plotkin and Lea Albaugh), a *website* (an index.html file), and an *interpreter*, which will allow our IF to be played

Boyd, *An Introductory Guide to Inform 7*

in a browser (if we upload it on, for example, ifdb.com or itch.io). If a website is included, an interpreter will be included automatically as well. We can also include a *solution* (or a *public solution*) generated from our IF's Skein (see 11.1), as well as the *source text*.

To include all these things, in the source text/code, we need to include them in a "Release along with" statement:

Release along with cover art ("[alt text description]"), a website, the introductory booklet, and the introductory postcard.

The text in parentheses and quotation marks after "cover art" functions as the alt text for the image (for the visually impaired).

To create a publishable version of our IF, we click on the "Release" button in the top left corner of the Inform IDE. For details about the release process, see WWI 25.6 "The Release button and the Materials folder".

Still to be included:

4.5 "Kinds of value", 4.7 "New either/or properties", 4.8 "New value properties", 4.9 "Using new kinds of value in properties", 4.10 "Conditions of things", 4.11 "Default values of kinds", 4.12 "Values that vary", 4.13 "Values that never vary", 4.16 "Names made in assembly"

5.1 "Text with substitutions", 5.3 "Text which names things", 5.4 "Text with numbers", 5.5 "Text with lists", 5.7 "Text with random alternatives", 5.13 "Making new substitutions"

7.4 "Try and Try Silently"

8.18 "Randomness", 8.19 "Random choices of things"

14. Adaptive Text and Responses: 14.1 "Tense and narrative viewpoint"; 14.9 "Verbs as values"

16. Tables: 16.9 "Blank rows" example?